

```
#!/usr/bin/env python3
```

```
"""
```

```
build_master_workbook.py
```

Consolidates every per-country "<Country>_University_PhD_LinkedIn_Postings.xlsx" and "<Country>_University_Masters_Funded_LinkedIn_Postings.xlsx" workbook in a folder into ONE Excel file, with one sheet per country/round plus a Master Summary dashboard sheet on top.

This is a pure LOCAL FILE tool: it never touches LinkedIn, never needs a password, and is safe to run as often as you like (it just re-reads whatever per-country .xlsx files already exist in the folder). Run it after you (or Claude) add or refresh a country's workbook.

Each run also compares the new row counts against whatever the master workbook already contained, and prints how many entries there were before, how many are new this run, and the new grand total. Any genuinely new postings (identified by LinkedIn Post Link, so insertions/reordering are handled correctly, not just appends) are additionally saved as their own snapshot file per country/round in a "New_Entries" subfolder, named with the country, round, and today's date - e.g. New_Entries/India_PhD_New_Entries_2026-09-17.xlsx - so you can check exactly what's new without opening the full master workbook.

Files currently open in Excel (which leave a temporary "~\$..." lock file next to them) are automatically skipped rather than crashing the run; close the file in Excel and re-run if you see it listed as skipped.

Usage (from Command Prompt):

```
python build_master_workbook.py
```

```
python build_master_workbook.py "D:\\LinkedInSunanda"
```

If no folder is given, it uses the folder this script lives in.

See runner.txt in this folder for a plain-language how-to.

```
"""
```

```
import sys
```

```
import glob
```

```
import os
```

```
import re
```

```
import datetime
```

```
import openpyxl
```

```
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
```

```
THIN = Side(style="thin", color="FFD9D9D9")
```

```
BORDER = Border(left=THIN, right=THIN, top=THIN, bottom=THIN)
```

```
CS_FILL = PatternFill("solid", fgColor="FFDDEBF7")
```

```
BIZ_FILL = PatternFill("solid", fgColor="FFFCE4D6")
```

```
PHD_THEME = "FF1F4E78"
```

```
MASTERS_THEME = "FF1F6E51"
```

```
CANON_HEADERS = ["No.", "University / Institution", "Location", "Department / Lab", "Category",  
                 "Subfield", "Position Summary", "Start / Deadline", "Contact",  
                 "LinkedIn Post Link", "Posted By", "Source Type", "Posted"]
```

```
COL_WIDTHS = {"A": 5.0, "B": 30.0, "C": 14.0, "D": 32.0, "E": 16.0, "F": 26.0,  
              "G": 48.0, "H": 22.0, "I": 26.0, "J": 24.0, "K": 22.0, "L": 22.0, "M": 9.0}
```

```
PHD_SUFFIX = "_University_PhD_LinkedIn_Postings.xlsx"
```

```
MASTERS_SUFFIX = "_University_Masters_Funded_LinkedIn_Postings.xlsx"
```

```
OUTPUT_NAME = "All_Countries_Funded_PhD_Masters_Master.xlsx"
```

```
NEW_ENTRIES_SUBFOLDER = "New_Entries"
```

```
LINK_COL_INDEX = 9 # 0-based index of "LinkedIn Post Link" in CANON_HEADERS
```

```
def is_temp_or_lock_file(path):
```

```
    """True for Excel/Office lock files (~$Name.xlsx) and OS junk (.DS_Store-style  
    dotfiles) that can otherwise accidentally match our glob patterns."""
```

```
    base = os.path.basename(path)
```

```
    return base.startswith("~$") or base.startswith(".")
```

```
def discover_source_files(folder):
```

```
    """Return list of (country, round_label, theme, filepath), sorted by country then round."""
```

```

found = []
for path in glob.glob(os.path.join(folder, f"*{PHD_SUFFIX}")):
    if is_temp_or_lock_file(path):
        continue
    country = os.path.basename(path)[: -len(PHD_SUFFIX)].replace("_", " ")
    found.append((country, "PhD", PHD_THEME, path))
for path in glob.glob(os.path.join(folder, f"*{MASTERS_SUFFIX}")):
    if is_temp_or_lock_file(path):
        continue
    country = os.path.basename(path)[: -len(MASTERS_SUFFIX)].replace("_", " ")
    found.append((country, "Masters", MASTERS_THEME, path))
found.sort(key=lambda t: (t[0], t[1]))
return found

```

```

def safe_sheet_name(name, used):
    name = re.sub(r'[\\"/*\[]:~?]', "", name)[:31]
    base, i = name, 1
    while name in used:
        suffix = f"({i})"
        name = base[:31 - len(suffix)] + suffix
        i += 1
    used.add(name)
    return name

```

```

def safe_filename_part(name):
    name = re.sub(r'[\\"/*\[]:~?<>|]', "", name).strip()
    name = re.sub(r"\s+", "_", name)
    return name or "Unknown"

```

```

def read_source_rows(path):
    """Read data rows (list of 13 values) from a source workbook's 'All Postings' sheet."""
    wb = openpyxl.load_workbook(path, data_only=True)
    if "All Postings" not in wb.sheetnames:
        return []
    ws = wb["All Postings"]
    rows = []
    for r in range(2, ws.max_row + 1):
        row_vals = [ws.cell(row=r, column=c).value for c in range(1, 14)]
        if all(v is None for v in row_vals):
            continue
        rows.append(row_vals)
    return rows

```

```

def read_previous_data(out_path):
    """Return {sheet_name: [row_lists]} for each per-country sheet already present
    in an existing master workbook, or {} if there isn't one yet / it can't be read."""
    if not os.path.isfile(out_path):
        return {}
    try:
        wb = openpyxl.load_workbook(out_path, data_only=True, read_only=True)
    except Exception:
        return {}
    data = {}
    for name in wb.sheetnames:
        if name == "Master Summary":
            continue
        ws = wb[name]
        rows = []
        for r in range(2, (ws.max_row or 1) + 1):
            row_vals = [ws.cell(row=r, column=c).value for c in range(1, 14)]
            if all(v is None for v in row_vals):
                continue
            rows.append(row_vals)
        data[name] = rows
    wb.close()
    return data

```

```

def row_key(row):
    """Identity key for a posting: its LinkedIn Post Link when present (so a
    posting is recognized even if inserted/reordered, not just appended),
    otherwise the full row content as a fallback."""
    link = row[LINK_COL_INDEX] if len(row) > LINK_COL_INDEX else None
    if link not in (None, ""):
        return ("link", link)
    return ("full", tuple(row))

def find_new_rows(current_rows, previous_rows):
    prev_keys = {row_key(r) for r in previous_rows}
    return [r for r in current_rows if row_key(r) not in prev_keys]

def write_country_sheet(wb, sheet_name, theme_hex, rows):
    ws = wb.create_sheet(sheet_name)
    ws.freeze_panes = "A2"
    header_font = Font(name="Arial", size=11, bold=True, color="FFFFFFFF")
    header_fill = PatternFill("solid", fgColor=theme_hex)
    header_align = Alignment(horizontal="center", vertical="center", wrap_text=True)
    data_align = Alignment(horizontal="left", vertical="top", wrap_text=True)
    data_font = Font(name="Arial", size=10)

    for col_idx, htext in enumerate(CANON_HEADERS, start=1):
        c = ws.cell(row=1, column=col_idx, value=htext)
        c.font, c.fill, c.alignment, c.border = header_font, header_fill, header_align, BORDER
    ws.row_dimensions[1].height = 15

    for i, row_vals in enumerate(rows, start=2):
        category = row_vals[4]
        fill = CS_FILL if category == "Computer Science" else BIZ_FILL
        for col_idx, val in enumerate(row_vals, start=1):
            c = ws.cell(row=i, column=col_idx, value=val)
            c.font, c.fill, c.alignment, c.border = data_font, fill, data_align, BORDER
        ws.row_dimensions[i].height = 45.75

    for col, width in COL_WIDTHS.items():
        ws.column_dimensions[col].width = width
    return len(rows)

def write_new_entries_file(folder, country, round_label, theme_hex, new_rows, date_str):
    subfolder = os.path.join(folder, NEW_ENTRIES_SUBFOLDER)
    os.makedirs(subfolder, exist_ok=True)
    filename = f"{safe_filename_part(country)}_{round_label}_New_Entries_{date_str}.xlsx"
    out_path = os.path.join(subfolder, filename)

    wb = openpyxl.Workbook()
    wb.remove(wb.active)
    write_country_sheet(wb, "New Postings", theme_hex, new_rows)
    wb.save(out_path)
    return out_path

def format_delta(delta):
    if delta > 0:
        return f"+{delta}"
    if delta < 0:
        return str(delta)
    return "+0"

def build(folder):
    sources = discover_source_files(folder)
    if not sources:
        print(f"No '{PHD_SUFFIX}' or '{MASTERS_SUFFIX}' files found in: {folder}")
        return None

    out_path = os.path.join(folder, OUTPUT_NAME)
    previous_data = read_previous_data(out_path)
    previous_counts = {name: len(rows) for name, rows in previous_data.items()}

```

```

previous_total = sum(previous_counts.values())
today_str = datetime.date.today().isoformat()

wb = openpyxl.Workbook()
wb.remove(wb.active)
summary_ws = wb.create_sheet("Master Summary", 0)

used_names = set()
sheet_refs = [] # (country, round_label, sheet_name, n_rows, prev_n, delta)
new_entry_files = [] # (country, round_label, path, count)
skipped = [] # (basename, reason)

for country, round_label, theme, path in sources:
    sheet_name = safe_sheet_name(f"{country} {round_label}", used_names)

    try:
        rows = read_source_rows(path)
    except Exception as e:
        skipped.append((os.path.basename(path), str(e)))
        print(f" ! Skipped {os.path.basename(path)} - couldn't read it ({e}). "
              f"If it's open in Excel, close it and re-run.")
        continue

    n = write_country_sheet(wb, sheet_name, theme, rows)

    prev_rows = previous_data.get(sheet_name, [])
    prev_n = len(prev_rows)
    delta = n - prev_n
    sheet_refs.append((country, round_label, sheet_name, n, prev_n, delta))
    print(f" + {sheet_name:<28} {n:>3} rows    (was {prev_n:>3}, {format_delta(delta):>3})"
          <- {os.path.basename(path)}")

    new_rows = find_new_rows(rows, prev_rows)
    if new_rows:
        new_path = write_new_entries_file(folder, country, round_label, theme, new_rows,
today_str)
        new_entry_files.append((country, round_label, new_path, len(new_rows)))
        print(f"      -> {len(new_rows)} new posting(s) saved to "
              f"{NEW_ENTRIES_SUBFOLDER}/{os.path.basename(new_path)}")

    if not sheet_refs:
        print("\nNothing could be read (all matching files were skipped) - nothing was
rebuilt.")
        return None

new_total = sum(item[3] for item in sheet_refs)
new_entries = new_total - previous_total

# ---- Master Summary sheet ----
summary_ws.column_dimensions["A"].width = 22
summary_ws.column_dimensions["B"].width = 12
summary_ws.column_dimensions["C"].width = 12
summary_ws.column_dimensions["D"].width = 18
summary_ws.column_dimensions["E"].width = 16

title = summary_ws.cell(row=1, column=1, value="All Countries - Funded PhD & Masters
LinkedIn Postings")
title.font = Font(name="Arial", size=16, bold=True, color="FF1F4E78")
summary_ws.merge_cells("A1:E1")

stamp = summary_ws.cell(
    row=2, column=1,
    value=(f"Consolidated {today_str} from {len(sheet_refs)} source workbook(s). "
          f"Previous total: {previous_total} | New this run: {format_delta(new_entries)}"
          "
          f"Total now: {new_total}"))
stamp.font = Font(name="Arial", size=10, color="FF595959")
summary_ws.merge_cells("A2:E2")

headers = ["Country", "Round", "Total", "Computer Science", "MBA/Business"]
header_font = Font(name="Arial", size=11, bold=True, color="FFFFFFFF")
header_fill = PatternFill("solid", fgColor="FF1F4E78")

```

```

for col_idx, h in enumerate(headers, start=1):
    c = summary_ws.cell(row=4, column=col_idx, value=h)
    c.font, c.fill, c.border = header_font, header_fill, BORDER
    c.alignment = Alignment(horizontal="center", vertical="center")

r = 5
for country, round_label, sheet_name, n, prev_n, delta in sheet_refs:
    last_row = n + 1
    ref = f"'{sheet_name}'" if " " in sheet_name or any(ch in sheet_name for ch in "-")
else sheet_name
    summary_ws.cell(row=r, column=1, value=country).font = Font(name="Arial", size=10)
    summary_ws.cell(row=r, column=2, value=round_label).font = Font(name="Arial", size=10)
    if n == 0:
        summary_ws.cell(row=r, column=3, value=0).font = Font(name="Arial", size=10,
bold=True)
        summary_ws.cell(row=r, column=4, value=0).font = Font(name="Arial", size=10)
        summary_ws.cell(row=r, column=5, value=0).font = Font(name="Arial", size=10)
    else:
        summary_ws.cell(row=r, column=3,
                        value=f'=COUNTA({ref}!A2:A{last_row})').font = Font(name="Arial",
size=10, bold=True)
        summary_ws.cell(row=r, column=4,
                        value=f'=COUNTIF({ref}!E2:E{last_row}, "Computer Science")').font
= Font(name="Arial", size=10)
        summary_ws.cell(row=r, column=5,
                        value=f'=COUNTIF({ref}!E2:E{last_row}, "MBA/Business")').font =
Font(name="Arial", size=10)
        for col_idx in range(1, 6):
            summary_ws.cell(row=r, column=col_idx).border = BORDER
        r += 1

total_row = r
summary_ws.cell(row=total_row, column=1, value="GRAND TOTAL").font = Font(name="Arial",
size=10, bold=True)
summary_ws.cell(row=total_row, column=3, value=f'=SUM(C5:C{total_row - 1})').font =
Font(name="Arial", size=10, bold=True)
summary_ws.cell(row=total_row, column=4, value=f'=SUM(D5:D{total_row - 1})').font =
Font(name="Arial", size=10, bold=True)
summary_ws.cell(row=total_row, column=5, value=f'=SUM(E5:E{total_row - 1})').font =
Font(name="Arial", size=10, bold=True)
for col_idx in range(1, 6):
    summary_ws.cell(row=total_row, column=col_idx).border = BORDER

note_row = total_row + 2
note = summary_ws.cell(row=note_row, column=1,
                        value=("Source: each row is generated from Claude's LinkedIn
content-search sweeps, one workbook "
                               "per country/round. This master file only re-reads those
local .xlsx files - it does not "
                               "access LinkedIn itself. Re-run build_master_workbook.py
after adding or refreshing a "
                               "country's workbook to rebuild this file. Newly identified
postings are also saved per "
                               "country/round into the New_Entries subfolder, dated the day
they were found.))
note.font = Font(name="Arial", size=8.5, color="FF595959")
note.alignment = Alignment(wrap_text=True)
summary_ws.merge_cells(start_row=note_row, start_column=1, end_row=note_row, end_column=5)
summary_ws.row_dimensions[note_row].height = 30

wb.save(out_path)
return {
    "out_path": out_path,
    "previous_total": previous_total,
    "new_entries": new_entries,
    "new_total": new_total,
    "sheet_refs": sheet_refs,
    "new_entry_files": new_entry_files,
    "skipped": skipped,
}

```

```
if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else os.path.dirname(os.path.abspath(__file__))
    print(f"Scanning: {folder}")
    result = build(folder)
    if result:
        print()
        print(f"Previous total entries : {result['previous_total']}")
        print(f"New entries this run      : {format_delta(result['new_entries'])}")
        print(f"Total entries now                : {result['new_total']}")
        if result["new_entry_files"]:
            print()
            print("New-entry snapshot files saved:")
            for country, round_label, path, count in result["new_entry_files"]:
                print(f"  - {country} {round_label}: {count} new posting(s) -> {path}")
            print(f"\nSaved: {result['out_path']}")
```